# Documento de Arquitetura de Software
# Sistema Co-pilot Médico
# 2025

# Histórico da Revisão

| Data | Versão | Descrição | Autor |
|---|---|---|---|
| 16/06/2025 | 2.0 | Botão Flutuante | Adelmo Neto/Jônatas |
| 06/05/2025 | 2.0 | Chat | Adelmo Neto/Jônatas |
| 20/05/2025 | 2.0 | Leitura de dados de arquivos PDF | Adelmo Neto/Jônatas |
| 13/05/2025 | 2.0 | Remoção do nome do paciente | Adelmo Neto/Jônatas |
| 10/06/2025 | 2.0 | Interpretação de áudio | Adelmo Neto/Jônatas |
| 03/06/2025 | 2.0 | Armazenamento de dados no banco de dados JSON | Adelmo Neto/Jônatas |

# Indice Analitico

**Histórico da Revisão** **2**
**Indice Analitico** **3**
**1. Introdução** **4**
**2. Requisitos:** **5**
**3.Visão Lógica** **5**
Estrutura de Camadas 5
1. Camada de Apresentação (Frontend - React): 6
2. Camada de Aplicação (Backend - Flask): 6
3. Camada de Dados (Backend): 6
Fluxo de Informação 7
Diagrama do fluxo 8
**4. Arquitetura:** **8**
Fluxo Frontend 9
Fluxo backend 9
**5. Diagrama de caso de uso** **10**
**6. Tecnologias:** **11**
Gerenciamento 11
Backend 11
Frontend 11

# 1. Introdução

Esse projeto foca no desenvolvimento de um sistema que possa facilitar a interação do médico com o paciente durante consultas médicas. Tem como funcionalidade inicial extrair dados do prontuário eletrônico Amplimed e enviar o que for extraído para API do sistema. Na atual versão foram acrescentadas as seguintes funcionalidades:

- Botão externo flutuante que realiza todo processo de extração sem a necessidade de abrir extensão.
- Chat direto através da  API
- Envio de arquivos PDF de forma a extrair dados deste formato de arquivo. Geralmente resultados de exames
- Destacar dados obtidos através da API para análise do médico auxiliando na tomada de decisão.
- Atualização do layout e funcionalidades do Front de forma a reconhecer Markdown
- Implementação de segurança e privacidade com a filtragem de dados como nome do paciente

- Histórico do diálogo no chat onde o usuário retorna a conversa do ponto que parou mesmo quando o sistema é minimizado
- Sistema inicial de ID e banco de dados em JSON para histórico de conversa.

Foi construído como projeto proposto na disciplina de Engenharia de software da Universidade de Pernambuco. Tem como público alvo médicos e pacientes.

# 2. Requisitos:

- Leitura do prontuário eletrônico Amplimed extraindo dados para ferramenta
- Upload de PDF de exames médicos
- Ocultar nomes do paciente
- Armazenamento de dados em banco local
- Destacar dados relevantes de exames

# 3.Visão Lógica

A arquitetura do sistema é dividida em duas camadas principais: o Frontend (Cliente), que opera como uma extensão do Google Chrome, e o Backend (Servidor), que processa as requisições e se comunica com a API de IA. O fluxo de informações é projetado para ser seguro e contextual, garantindo a privacidade dos dados do paciente.

## Estrutura de Camadas

1. Camada de Apresentação (Frontend - React):

- Componentes da Interface (chat.tsx, App.tsx): Responsável por toda a interação com o usuário, incluindo a exibição do chat, botões de upload e o formulário de entrada de texto.
- Scripts de Conteúdo (content.js): Injetado em páginas web (especificamente no prontuário eletrônico Amplimed) para extrair dados relevantes diretamente da fonte.
- Gerenciamento de Estado e Sessão (App.tsx, background.js): Controla a sessão do paciente (patient_id) e o histórico de mensagens, utilizando o chrome.storage.local para persistência no navegador.

2. Camada de Aplicação (Backend - Flask):

- API Gateway (server.py): Atua como o ponto de entrada único para o frontend. Ele recebe as requisições, orquestra as chamadas para outros módulos e formata as respostas.
- Lógica de Negócio (gemini_connection.py, text_filter.py): Onde a "mágica" acontece. Este núcleo processa a lógica de negócio: anonimiza os dados do usuário removendo nomes, gerencia o histórico de conversas para dar contexto à IA e se comunica com a API do Google Gemini.
- Processadores de Dados (pdf_reader.py, audio_listener.py): Módulos especializados que lidam com a conversão de diferentes formatos de dados (PDFs, áudio) em texto simples que pode ser processado pela IA.

3. Camada de Dados (Backend):

Persistência (patient_db.py): Responsável por salvar e recuperar o histórico de conversas de cada paciente. Utiliza um arquivo JSON (patients_db.json) como um banco de dados simples para armazenar as sessões de forma estruturada.

## Fluxo de Informação

O fluxo de informação é sempre iniciado pelo usuário no frontend e segue um padrão cliente-servidor:

1. Usuário interage com a extensão (envia uma mensagem, faz upload de um PDF ou clica em "Extrair Dados").
2. O Frontend (React) captura a ação e envia uma requisição fetch para o endpoint apropriado no Backend (Flask), juntamente com o patient_id da sessão atual.
3. O Backend (server.py) recebe a requisição.
4. Os dados são processados e anonimizados (text_filter.py).
5. A mensagem tratada, junto com o histórico recuperado do banco de dados JSON (patient_db.py), é enviada para a API do Google Gemini (gemini_connection.py).
6. A resposta da IA é recebida pelo backend.
7. O backend salva a nova interação (pergunta do usuário e resposta da IA) no banco de dados JSON.
8. A resposta é enviada de volta ao Frontend, que a exibe para o usuário.

# Diagrama do fluxo

# 4. Arquitetura:

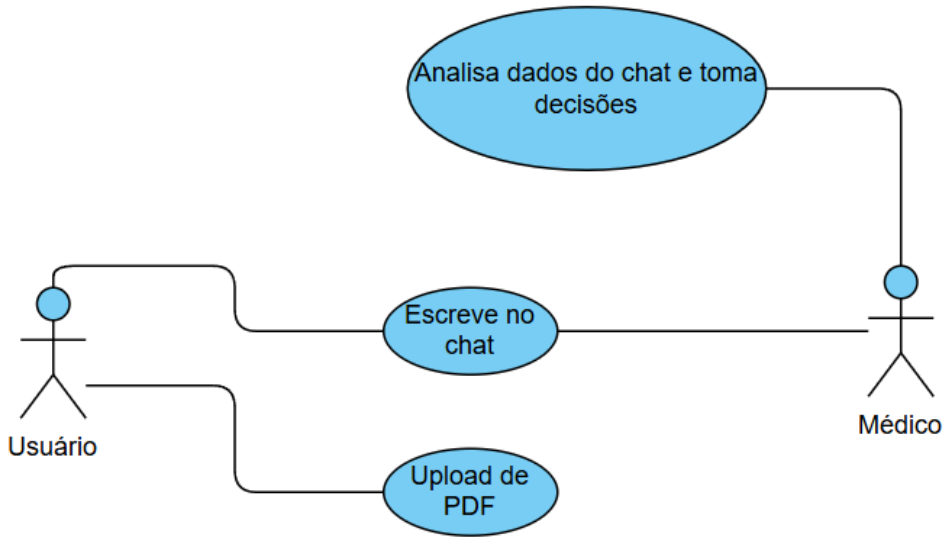
## Fluxo Frontend

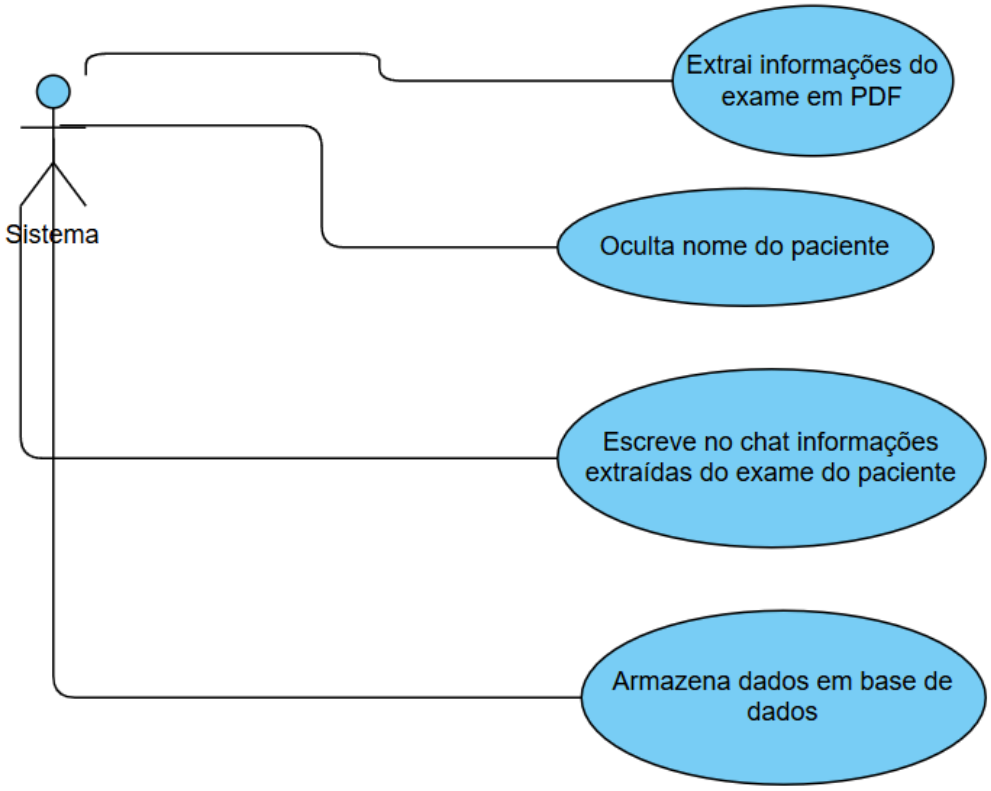
1. Médico/paciente acessa o Chatbot utilizando extensão no navegador
2. Sistema recebe informações através do chat ou arquivo pdf
3. Dados recebidos são enviados para o servidor backend

## Fluxo backend

1. Servidor recebe dados enviados pelo frontend
2. Nome do paciente é removido
3. Dados são enviados para base de dados
4. Os dados são analisados e as informações retornam ao frontend
5. Frontend apresenta dados extraídos no chat para análise do médico

# 5. Diagrama de caso de uso



# 6. Tecnologias:

O desenvolvimento do sistema foi estruturado em duas camadas principais: frontend e backend, com o uso de tecnologias modernas que oferecem desempenho, escalabilidade e produtividade. A seguir, são apresentadas as tecnologias selecionadas para cada camada, bem como as ferramentas complementares utilizadas no projeto

## Gerenciamento

- Trello: Ferramenta para gestão visual de projetos e tarefas
- Jira: Ferramenta de gestão de projetos e rastreamento de problemas

## Backend

- Python (3.11.2): linguagem de programação de alto nível, interpretada, orientada a objetos
- Flask (3.0.3): microframework web escrito em Python
- API Gemini: interface de programação de aplicativos (API) desenvolvida pelo Google que permite aos desenvolvedores acessar e integrar os modelos de linguagem do Gemini em seus aplicativos
- spaCy: biblioteca gratuita e de código aberto para Processamento de Linguagem Natural em Python
- PyPDF2: ferramenta versátil que permite dividir, juntar, recortar, transformar, criptografar e descriptografar arquivos PDF
- JSON:

## Frontend

- JavaScript (v18.12.1): linguagem de programação usada principalmente para adicionar interatividade e dinamismo a páginas web
- Extensão Chrome ("manifest_version" : 3)
- Node: é um ambiente de execução de código JavaScript de código aberto e multiplataforma
- React: biblioteca JavaScript de código aberto para construir interfaces de usuário (UI) interativas